생성형 AI 오케스트레이션: ComfyUI와 오픈소스 모델을 활용한 콘텐츠 생성 실습

강의노트

동서대학교 소프트웨어학과 이상걸 (leesg@dongseo.ac.kr)

목차

1. ComfyUI 개요
2. 수치 정밀도 (Numerical Precision)
3. 모델 파일 형식
4. 텍스트 인코더 (Text Encoders)
5. VAE (Variational Autoencoder)
6. 양자화 (Quantization)
7. 주요 생성 모델 아키텍처
8. LoRA와 파인튜닝
9. 실습 팁과 워크플로우 구성

---

1. ComfyUI 개요

1.1 ComfyUI란?

- **노드 기반 워크플로우 시스템**: 시각적 프로그래밍 방식으로 생성형 AI 파이프라인 구축
- **크로스 플랫폼**: Windows, macOS, Linux 지원
- **확장성**: 커스텀 노드 개발 가능
- **로컬 실행**: 개인 하드웨어에서 완전한 제어권 확보

1.2 2026년 최신 버전 주요 기능

(2026년 2월 기준)

Version 0.14.2 주요 업데이트:

- **Gemma 12B 지원**: 양자화 가중치 지원
- **LTXAV 텍스트 인코더**: 디바이스 선택 및 VRAM 사용량 감소
- **FP8MM 오프로딩**: 성능 개선

- **통합 Jobs API**: `/api/jobs` 엔드포인트를 통한 워크플로우 모니터링

모델 지원 확장:

- **이미지 생성**: Qwen Image, Z-Image Fun Control, Kandinsky 5.0
- **비디오 생성**: Wan2.6, Kling Omni (텍스트→비디오, 이미지→비디오+오디오)
- **3D 모델**: Tripo3.0
- **오디오**: 네이티브 오디오 녹음 노드, 오디오-비디오 통합

성능 최적화:

- AMD GPU 지원 강화 (FP8 연산 활성화)
- Apple Silicon FP16 어텐션 문제 해결
- 서브그래프 위젯 편집 기능
- 템플릿 모달 재설계 (모델 태그 및 카테고리별 필터링)

1.3 디렉토리 구조

```
ComfyUI/
├── models/
│   ├── checkpoints/        # 전체 모델 파일 (.safetensors, .ckpt)
│   ├── unet/                # UNet 컴포넌트만 있는 모델
│   ├── vae/                 # VAE 인코더/디코더
│   ├── text_encoders/       # CLIP, T5 등 텍스트 인코더
│   │   ├── clip_g.safetensors
│   │   ├── clip_l.safetensors
│   │   ├── t5xxl_fp8_e4m3fn.safetensors
│   │   └── t5xxl_fp16.safetensors
│   ├── loras/               # LoRA 어댑터 파일
│   └── controlnet/          # ControlNet 모델
```

파일 배치 규칙:

- CLIP 컴포넌트가 **포함된** 모델: `checkpoints/` 폴더
- CLIP 컴포넌트가 **없는** 모델: `unet/` 폴더
- 텍스트 인코더 (T5 포함): `text_encoders/` 또는 `clip/` 폴더

---

2. 수치 정밀도 (Numerical Precision)

2.1 기본 개념

수치 정밀도는 모델의 가중치와 활성화 값을 표현하는 비트 수를 의미합니다. 정밀도가 높을수록 정확도는 높아지지만, 메모리 사용량과 계산 비용이 증가합니다.

2.2 부동소수점 형식 비교

FP64 (Double Precision Floating Point)

- **비트 구성**: 64비트
- **정밀도**: 매우 높음 (약 15-17 자릿수)
- **용도**: 과학 계산, 정밀한 수치 시뮬레이션
- **AI 사용**: 거의 사용되지 않음 (오버킬)

FP32 (Single Precision Floating Point)

- **비트 구성**: 32비트 (부호 1비트 + 지수 8비트 + 가수 23비트)
- **정밀도**: 높음 (약 6-9 자릿수)
- **동적 범위**: $\pm 3.4 \times 10^{38}$
- **용도**: 일반적인 딥러닝 학습 및 추론
- **메모리**: 모델 파라미터당 4바이트
- **특징**: 안정성과 정확도의 균형

FP16 (Half Precision Floating Point / IEEE 754 Half Precision)

- **비트 구성**: 16비트 (부호 1비트 + 지수 5비트 + 가수 10비트)
- **정밀도**: 중간 (약 3-4 자릿수)
- **동적 범위**: ±65,504 (매우 제한적)
- **장점**:
  - FP32 대비 50% 메모리 절감
  - 더 높은 수치 정밀도 ($2^{10} \approx 1024$ 단계)
  - 추론 시 세밀한 표현 가능
- **단점**:
  - 학습 시 그래디언트 언더플로우/오버플로우 위험
  - ±65,000 범위 초과 시 스케일링 필요
- **용도**: 추론, 혼합 정밀도 학습
- **메모리**: 모델 파라미터당 2바이트

BF16 (Brain Floating Point 16)

- **비트 구성**: 16비트 (부호 1비트 + 지수 8비트 + 가수 7비트)
- **정밀도**: 낮음 (약 2-3 자릿수)
- **동적 범위**: FP32와 동일 ($\pm 3.4 \times 10^{38}$)
- **장점**:
  - FP32와 같은 지수 범위 → 수치 안정성 우수
  - 그래디언트 스케일링 불필요
  - 학습 시 더 안정적

    - 더 작은 수 표현 가능 (9.18e-41까지)
- **단점**:
    - FP16보다 낮은 정밀도 ($2^7 \approx 128$ 단계)
    - 0에 가까운 값은 0으로 반올림될 위험
- **용도**: 대규모 언어 모델 학습 (TPU, 최신 GPU)
- **메모리**: 모델 파라미터당 2바이트
- **주요 사용**: Google TPU, NVIDIA A100 이상

TF32 (TensorFloat-32)

- **비트 구성**: 내부적으로 19비트 (부호 1비트 + 지수 8비트 + 가수 10비트)
- **특징**:
    - NVIDIA Ampere 아키텍처(A100, RTX 3090 이상) 전용
    - Tensor Core 연산에서 자동 사용
    - FP32 코드를 수정하지 않고 가속
    - 저장은 FP32, 연산은 TF32
- **용도**: 딥러닝 학습 가속

FP8 (8-bit Floating Point)

- **비트 구성**: 8비트 (여러 변형 존재)
    - **E4M3**: 지수 4비트 + 가수 3비트
    - **E5M2**: 지수 5비트 + 가수 2비트
- **장점**:
    - FP16 대비 50% 추가 메모리 절감
    - 최신 GPU(H100, H200)에서 하드웨어 가속
- **단점**: 매우 낮은 정밀도
- **용도**: 대규모 모델 추론, 최첨단 학습

INT8 (8-bit Integer)

- **비트 구성**: 8비트 정수
- **범위**: -128 ~ 127 (signed) 또는 0 ~ 255 (unsigned)
- **용도**: 양자화된 모델 추론
- **장점**: 매우 빠른 연산 속도, 적은 메모리
- **단점**: 정밀도 손실

2.3 정밀도 비교 표

형식	비트	지수	가수	동적 범위	정밀도	메모리 (13B 모델)	주요 용도	상대 속도
FP64	64	11	52	$\pm 1.8\times10^{308}$	최고	104 GB	과학 계산	0.5x

형식	비트	지수	가수	동적 범위	정밀도	메모리 (13B 모델)	주요 용도	상대 속도
FP32	32	8	23	$\pm 3.4\times10^{38}$	높음	52 GB	기본 학습/추론	1x
TF32	19*	8	10	$\pm 3.4\times10^{38}$	중상	52 GB (저장)	Ampere+ 학습	1.5-2x
BF16	16	8	7	$\pm 3.4\times10^{38}$	중하	26 GB	LLM 학습	2x
FP16	16	5	10	±65,504	중상	26 GB	추론, 혼합 정밀 학습	2x
FP8	8	4-5	2-3	제한적	낮음	13 GB	최첨단 추론	3-4x
INT8	8	-	-	-128~127	낮음	13 GB	양자화 추론	3-4x

*TF32는 내부 연산 형식

2.4 파일명 표기 예시

- `flux_dev_fp16.safetensors`: FP16 정밀도
- `sd3.5_large_bf16.safetensors`: BF16 정밀도
- `t5xxl_fp8_e4m3fn.safetensors`: FP8 E4M3 정밀도
- `umt5-xxl-enc-bf16.safetensors`: BF16 인코더

2.5 실무 선택 가이드

학습 (Training):

- 일반적: FP32 또는 혼합 정밀도 (FP32 + FP16/BF16)
- 대규모 LLM: BF16 (안정성 우수)
- 최신 GPU (A100+): TF32 자동 활성화

추론 (Inference):

- 품질 우선: FP16 (24배 더 높은 수치 정밀도 vs BF16)
- 메모리 제약: INT8 양자화 또는 FP8
- 균형: BF16 또는 FP16

ComfyUI에서:

- SDXL/SD3: BF16 또는 FP16
- Flux: FP16 권장 (프롬프트 이해도 향상)
- T5 인코더: FP8 또는 FP16 (메모리 vs 품질 트레이드오프)

---

3. 모델 파일 형식

3.1 Safetensors (.safetensors)

개요

- **개발**: Hugging Face
- **형식**: 바이너리 파일 형식
- **보안**: 임의 코드 실행 방지 (pickle 취약점 해결)
- **속도**: 빠른 로딩 및 저장
- **메모리 안전**: 메모리 매핑(mmap) 지원

구조

```
┌──────────────────────────┐
│    메타데이터 헤더       │  ← JSON 형식, 모델 정보
├──────────────────────────┤
│    텐서 메타데이터       │  ← 텐서 이름, 크기, dtype, 오프셋
├──────────────────────────┤
│      텐서 데이터         │  ← 실제 가중치 데이터
└──────────────────────────┘
```

장점

- ✅ **보안**: .ckpt 파일의 pickle 취약점 없음
- ✅ **빠른 로딩**: 메모리 매핑으로 전체 파일 로드 불필요
- ✅ **크로스 플랫폼**: Python, Rust, JavaScript에서 사용 가능
- ✅ **호환성**: PyTorch, TensorFlow, JAX 지원

단점

- ❌ Python 객체 직렬화 불가 (오직 텐서만)

파일명 예시

```
sd_xl_base_1.0.safetensors          # SDXL 기본 모델
flux1-dev-fp16.safetensors          # Flux Dev FP16
sdxl_vae.safetensors                # SDXL VAE
clip_l.safetensors                  # CLIP-L 텍스트 인코더
t5xxl_fp8_e4m3fn.safetensors        # T5-XXL FP8 인코더
```

3.2 Checkpoint (.ckpt, .pt, .pth)

개요

- **형식**: PyTorch pickle 기반
- **역사적 표준**: 이전 Stable Diffusion 모델에 널리 사용
- **보안 우려**: 임의 코드 실행 가능

장점

- ✅ 복잡한 Python 객체 저장 가능
- ✅ 레거시 호환성

단점

- ❌ **보안 위험**: 악성 코드 포함 가능
- ❌ 느린 로딩 속도
- ❌ 큰 파일 크기

권장사항

⚠️ **ComfyUI에서는 .safetensors 사용을 강력히 권장**

3.3 GGUF (GPT-Generated Unified Format)

개요

- **개발**: llama.cpp 프로젝트 (Georgi Gerganov)
- **목적**: 빠른 로딩과 효율적인 추론을 위한 바이너리 형식
- **주요 사용**: 대규모 언어 모델 (LLM) 양자화 및 배포
- **후속 형식**: GGML, GGMF, GGJT의 개선판

구조

```
┌─────────────────────────┐
│  메타데이터 섹션          │  ← 키-밸류 쌍 (모델 아키텍처, 버전, 하이퍼파라미터)
├─────────────────────────┤
│  텐서 메타데이터 섹션     │  ← 텐서 이름, 크기, dtype, 오프셋
├─────────────────────────┤
│  텐서 데이터 섹션         │  ← 양자화된 가중치 데이터
└─────────────────────────┘
```

양자화 스킴 (Quantization Schemes)

GGUF의 핵심 특징은 유연한 양자화 지원입니다:

- **Q4_K_M**: 4비트, K-평균 클러스터링, 중간 블록 크기

- Q5_K_M: 5비트, K-평균 클러스터링, 중간 블록 크기
- Q8_0: 8비트, 기본 선형 양자화
- IQ4_XS: 4비트, 초소형 블록 (고급 압축)

장점

- ✅ **CPU 최적화**: 소비자 하드웨어에서 효율적 실행
- ✅ **빠른 로딩**: mmap 지원으로 초고속 모델 로드
- ✅ **단일 파일**: 모든 정보를 하나의 파일에 포함
- ✅ **확장성**: 메타데이터 추가 시 하위 호환성 유지
- ✅ **양자화**: 다양한 정밀도 옵션 (Q2~Q8)

단점

- ❌ **제한적 사용**: 주로 LLM, 디퓨전 모델은 덜 일반적
- ❌ **GPU 가속 제한**: CPU 추론에 최적화

지원 프레임워크

- llama.cpp
- Ollama
- vLLM
- GPT4All
- LM Studio

사용 예시

```
# Hugging Face 모델을 GGUF로 변환
python convert-hf-to-gguf.py --model-id meta-llama/Llama-3-8B

# Ollama에서 GGUF 모델 실행
ollama run llama3:8b-q4_k_m
```

GGUF 파일명 예시

```
llama-3-8b-q4_k_m.gguf                # Llama 3 8B, Q4_K_M 양자화
mistral-7b-instruct-q5_k_m.gguf       # Mistral 7B, Q5_K_M 양자화
phi-3-mini-q8_0.gguf                  # Phi-3 Mini, Q8_0 양자화
```

3.4 형식 비교 표

특성	Safetensors	Checkpoint (.ckpt)	GGUF
보안	✅ 안전	❌ 취약	✅ 안전
로딩 속도	⚡ 빠름	느림	⚡⚡ 매우 빠름
메모리 효율	✅ mmap 지원	❌ 전체 로드	✅✅ mmap 최적화
양자화 지원	부분적	부분적	✅✅ 네이티브
크로스 플랫폼	✅	❌	✅
주요 용도	디퓨전 모델	레거시	LLM
ComfyUI 권장	✅✅ 최우선	△ 호환성만	❌ 비지원*

*ComfyUI는 주로 디퓨전 모델용이므로 GGUF는 거의 사용되지 않음

3.5 파일 확장자 요약

- `.safetensors`: 최신 표준, 보안 우수 ✅
- `.ckpt`, `.pt`, `.pth`: 레거시 PyTorch 체크포인트 ⚠️
- `.gguf`: LLM 양자화 전용 (llama.cpp)
- `.bin`: 구형 Hugging Face 형식
- `.yaml`: 모델 설정 파일 (체크포인트와 함께 사용)

---

4. 텍스트 인코더 (Text Encoders)

4.1 텍스트 인코더의 역할

텍스트 인코더는 자연어 프롬프트를 디퓨전 모델이 이해할 수 있는 수치 벡터(임베딩)로 변환합니다.

```
"A cat sitting on a mat" → [0.23, -0.45, 0.89, ...] (임베딩 벡터)
                                    ↓
                              디퓨전 모델 조건화
                                    ↓
                              이미지 생성
```

4.2 CLIP (Contrastive Language-Image Pre-training)

개념

- **개발**: OpenAI (2021)
- **학습 방법**: 대조 학습 (이미지-텍스트 쌍 4억 개)

- **목적**: 텍스트와 이미지를 공유 임베딩 공간에 매핑

CLIP 변형 비교

CLIP-L (Large)

- **모델 크기**: ViT-L/14 (Large Vision Transformer)
- **파라미터**: 약 427M (4억 2700만)
- **임베딩 차원**: 768차원
- **토큰 제한**: 77 토큰 (실효 길이 ~20 토큰)
- **특징**:
  - 키워드 중심 이해
  - 간단한 프롬프트에 효과적
  - 낮은 메모리 사용량
- **사용 예**: Stable Diffusion 1.x, 2.x

CLIP-G (Giant)

- **모델 크기**: ViT-G/14 (Giant Vision Transformer)
- **파라미터**: 약 1.4B (14억)
- **임베딩 차원**: 1280차원
- **토큰 제한**: 77 토큰
- **특징**:
  - 더 균형잡힌 데이터 분포
  - 복잡한 프롬프트 이해 향상
  - CLIP-L보다 높은 표현력
- **사용 예**: Stable Diffusion XL (SDXL)

CLIP-H (Huge)

- **파라미터**: 약 632M (6억 3200만)
- **임베딩 차원**: 1024차원
- **특징**: CLIP-L과 CLIP-G 사이의 중간 크기
- **사용 예**: 일부 실험적 모델

CLIP 한계

⚠️ **77 토큰 제한**: 긴 상세한 프롬프트 처리 어려움 ⚠️ **실효 길이**: 20 토큰 이후 성능 저하 ⚠️ **문법 이해 부족**: 키워드 중심, 문장 구조 이해 약함

Long-CLIP

- **개선점**: 248 토큰까지 지원
- **방법**:

    - Knowledge-Preserved Stretching (위치 임베딩 확장)
    - Primary Component Matching (CLIP 특성 유지)
- **성능**: 긴 캡션 검색 25% 향상
- **호환성**: 기존 CLIP 교체 가능 (Stable Diffusion 플러그인)

4.3 T5 (Text-To-Text Transfer Transformer)

개념

- **개발**: Google Research (2019)
- **아키텍처**: 인코더-디코더 트랜스포머
- **학습 데이터**: C4 (Colossal Clean Crawled Corpus)
- **특징**: 자연어 모델링 (비전 학습 아님)

T5 변형 비교

T5-Small

- **파라미터**: 60M (6천만)
- **레이어**: 인코더 6층, 디코더 6층
- **임베딩 차원**: 512
- **용도**: 경량 실험

T5-Base

- **파라미터**: 220M (2억 2천만)
- **레이어**: 인코더 12층, 디코더 12층
- **임베딩 차원**: 768
- **특징**:
    - T5-XXL 대비 50배 작음
    - 시각적 매력과 의미 이해 97% 유지
    - 텍스트 렌더링에 효과적
- **메모리**: 매우 효율적

T5-Large

- **파라미터**: 770M (7억 7천만)
- **레이어**: 인코더 24층, 디코더 24층
- **임베딩 차원**: 1024

T5-XL

- **파라미터**: 3B (30억)

- **레이어**: 인코더 24층, 디코더 24층
- **임베딩 차원**: 2048
- **특징**:
  - T5-XXL 성능의 97% 유지
  - 4배 작은 크기
  - 실용적 선택

T5-XXL (Extra Extra Large)

- **Full name**: Text-To-Text Transfer Transformer - Extra Extra Large
- **파라미터**: 11B (110억)
- **레이어**: 인코더 24층, 디코더 24층
- **임베딩 차원**: 4096
- **피드포워드 차원**: 65,536
- **어텐션 헤드**: 128
- **특징**:
  - 복잡한 프롬프트 이해 우수
  - 긴 문장 구조 파악
  - 텍스트 렌더링 (이미지 내 글자 생성) 능력
  - 공간 관계 이해 (2D, 3D)
  - 숫자 개념 (numeracy)
- **단점**:
  - 높은 VRAM 요구량 (FP16: ~22GB)
  - 느린 인코딩 속도
- **사용 예**:
  - Google Imagen
  - Stable Diffusion 3 (SD3)
  - Flux

T5-XXL 메모리 최적화 전략

1. **FP8 양자화**: `t5xxl_fp8_e4m3fn.safetensors` (50% 메모리 절감)
2. **제거**: SD3에서 T5 제외 시 시각적 품질 유지, 타이포그래피만 약간 저하
3. **증류 (Distillation)**: T5-Base 사용 (품질 97% 유지, 크기 50배 감소)

4.4 UMT5 (Universal Multilingual T5)

개념

- **Full name**: Universal Multilingual Text-To-Text Transfer Transformer
- **개발**: Google Research
- **특징**: T5의 다국어 확장 버전

- **지원 언어**: 100개 이상 언어

UMT5-XXL

- **Full name**: Universal Multilingual T5 - Extra Extra Large
- **파라미터**: 11B (T5-XXL과 동일 규모)
- **임베딩 차원**: 4096
- **특징**:
  - 다국어 프롬프트 지원
  - 한국어, 중국어, 일본어 등 비영어권 언어 이해 우수
  - T5-XXL과 유사한 아키텍처

파일명 예시

```
umt5-xxl-enc-bf16.safetensors        # UMT5-XXL 인코더, BF16 정밀도
t5xxl_fp8_e4m3fn.safetensors         # T5-XXL, FP8 E4M3 양자화
t5xxl_fp16.safetensors               # T5-XXL, FP16 정밀도
```

4.5 텍스트 인코더 조합 전략

Stable Diffusion 1.5/2.x

```
CLIP-L (단독) → 768차원 임베딩
```

Stable Diffusion XL (SDXL)

```
CLIP-L (768차원) + CLIP-G (1280차원) → 풀링된 프롬프트 임베딩
```

- **장점**: 두 인코더의 강점 결합
- **CLIP-G**: 인간 언어 이해 (예: "beautiful woman with red eyes")
- **CLIP-L**: 키워드 스타일 (예: "cinematic lighting, blurry background")

Stable Diffusion 3 (SD3)

```
CLIP-L (768차원) + CLIP-G (1280차원) + T5-XXL (4096차원)
```

- **역할 분담**:

    - **CLIP 모델**: 시각적 개념, 스타일, 색상
    - **T5-XXL**: 복잡한 문법, 공간 관계, 텍스트 렌더링
- **메모리 절충**: T5 제외 가능 (시각적 품질 유지, 타이포그래피 저하)

Flux.1

```
CLIP-L (768차원) + T5-XXL (4096차원)
```

- **특징**: CLIP-G 제외, 간소화된 구조
- **강점**: T5-XXL의 강력한 언어 이해

4.6 텍스트 인코더 비교 표

인코더	파라미터	임베딩 차원	토큰 제한	강점	약점	주요 사용
CLIP-L	427M	768	77	키워드 이해, 빠름	긴 프롬프트 약함	SD 1.x, 2.x
CLIP-G	1.4B	1280	77	균형잡힌 이해	토큰 제한	SDXL, SD3
CLIP-H	632M	1024	77	중간 성능	토큰 제한	실험적
T5-Base	220M	768	512+	경량, 효율적	품질 3% 손실	증류 모델
T5-XL	3B	2048	512+	좋은 균형	중간 VRAM	효율 중시
T5-XXL	11B	4096	512+	최고 품질, 문법 이해	높은 VRAM	SD3, Flux, Imagen
UMT5-XXL	11B	4096	512+	다국어 지원	높은 VRAM	다국어 모델

---

5. VAE (Variational Autoencoder)

5.1 VAE의 역할

개념

VAE는 이미지와 잠재 공간(latent space) 사이를 변환하는 신경망입니다. 디퓨전 모델은 고해상도 픽셀 공간이 아닌 압축된 잠재 공간에서 작동합니다.

구조

```
이미지 (512×512×3)
    ↓ Encoder
잠재 표현 (64×64×4) ← 디퓨전 모델이 여기서 작동
    ↓ Decoder
재구성 이미지 (512×512×3)
```

5.2 VAE 컴포넌트

VAE Encoder

- **입력**: 픽셀 이미지 (RGB, 예: 512×512×3)
- **출력**: 잠재 표현 (예: 64×64×4)
- **압축 비율**: 일반적으로 8배 (8×8×1 픽셀 → 1×1×4 잠재)
- **역할**:
    - 이미지를 압축된 벡터 공간으로 인코딩
    - 노이즈 추가 및 변환 준비
- **사용 시점**: 이미지→잠재 변환 (Image-to-Image, Inpainting)

VAE Decoder

- **입력**: 잠재 표현 (예: 64×64×4)
- **출력**: 픽셀 이미지 (RGB, 예: 512×512×3)
- **역할**:
    - 잠재 공간의 압축된 데이터를 실제 이미지로 복원
    - 최종 생성 이미지 출력
- **사용 시점**: 생성 파이프라인의 마지막 단계

5.3 주요 VAE 모델

SD1.5 VAE

- **적용 모델**: Stable Diffusion 1.x, 2.x
- **잠재 차원**: 4 채널
- **압축 비율**: 8x
- **특징**: 기본 VAE, 가끔 색상 왜곡 발생

SDXL VAE

- **파일명**: `sdxl_vae.safetensors`
- **적용 모델**: Stable Diffusion XL (SDXL) 전용
- **특징**:

    - SDXL 잠재 분포에 최적화
    - SD1.5 VAE와 **호환 불가**
    - 더 나은 색상 정확도
- VRAM: 약 2.5GB
- **주의사항**:
    - ⚠️ 파일명 변경 금지 (.safetensors 확장자 유지)
    - ⚠️ 다중 VAE 로드 금지 (파이프라인 혼란)

SD3 VAE

- **적용 모델**: Stable Diffusion 3
- **개선사항**: 향상된 오토인코딩 모델
- **특징**: 더 나은 디테일 보존

5.4 ComfyUI에서 VAE 사용

VAE Encode 노드

- **카테고리**: `latent`
- **입력**:
    - `pixels` (IMAGE): 인코딩할 픽셀 이미지
    - `vae` (VAE): 사용할 VAE 모델
- **출력**:
    - `latent` (LATENT): 잠재 공간 표현
- **용도**: Image-to-Image, Inpainting 워크플로우

VAE Decode 노드

- **카테고리**: `latent`
- **입력**:
    - `samples` (LATENT): 디코딩할 잠재 데이터
    - `vae` (VAE): 사용할 VAE 모델
- **출력**:
    - `image` (IMAGE): 재구성된 픽셀 이미지
- **용도**: 생성 파이프라인의 최종 출력

VAE Loader 노드

- **설정 예시**:

```
{
  "id": 5,
  "type": "VAELoader",
```

```
    "properties": {
      "vae_name": "sdxl_vae.safetensors"
    }
  }
```

워크플로우 연결 예시

```
Checkpoint → KSampler → VAE Decode → SaveImage
                              ↑
                      VAELoader (sdxl_vae)
```

5.5 VAE 트러블슈팅

문제 1: VAE 로더에서 파일이 인식되지 않음

원인: 파일명 또는 확장자 변경 **해결**:

- 원본 파일명 복원
- `.safetensors` 확장자 확인
- `models/vae/` 폴더에 위치 확인

문제 2: 색상 왜곡 또는 artifact

원인: 잘못된 VAE 사용 (모델 불일치) **해결**:

- SDXL 모델 → `sdxl_vae.safetensors` 사용
- SD1.5/2.x 모델 → 해당 VAE 사용

문제 3: 메모리 부족

원인: VAE 디코딩 시 높은 VRAM 사용 **해결**:

- 배치 크기 감소
- 해상도 낮추기
- Tiled VAE 사용 (큰 이미지 분할 처리)

---

6. 양자화 (Quantization)

6.1 양자화란?

개념

양자화는 모델의 가중치와 활성화 값을 낮은 비트 정밀도로 변환하여 메모리 사용량과 연산량을 줄이는 압축 기법입니다.

예시

```
원본 FP32: 3.14159265359 (32비트)
    ↓ 양자화
INT8:      3 (8비트, 4배 압축)
```

6.2 양자화 스킴 (GGUF 기반)

Q2_K (2-bit K-quantization)

- **메모리 절감**: FP16 대비 ~75%
- **품질 저하**: 매우 심각 (30%+ 성능 손실)
- **속도**: 4.0-5.0배 빠름
- **용도**: 실험적, 극한 리소스 제약
- **권장**: ❌ 실용적이지 않음

Q3_K_M (3-bit K-quantization, Medium)

- **메모리 절감**: FP16 대비 ~70%
- **품질 저하**: 심각 (15-30% 성능 손실)
- **속도**: 3.5-4.5배 빠름
- **용도**: 매우 제한된 하드웨어
- **권장**: △ 소비자 하드웨어 최소 요구사항

Q4_K_M (4-bit K-quantization, Medium)

- **Full name**: 4-bit K-means Quantization - Medium Block Size
- **메모리 절감**: FP16 대비 ~50%
- **품질 저하**: 낮음 (5-15% 성능 손실, 대부분 모델에서 95% 품질 유지)
- **속도**: 3.0-4.0배 빠름
- **방법**:
  - 4비트 정밀도
  - K-평균 클러스터링으로 유사 가중치 그룹화
  - 중간 크기 블록 (밀집 레이어 최소 왜곡)
- **용도**: **가장 널리 사용되는 선택** (소비자 하드웨어 배포)
- **권장**: ✅✅ 대부분의 사용 사례에 최적

실용 예시:

```
Llama 2 13B:
- FP16: 32GB RAM 필요 (불가능 on 16GB PC)
- Q4_K_M: 12GB RAM 필요 (가능) + 2배 속도 향상 + 95% 품질
```

Q4_K_S (4-bit K-quantization, Small)

- **메모리 절감**: Q4_K_M과 유사
- **품질 저하**: Q4_K_M보다 약간 높음
- **블록 크기**: 더 작음 (더 공격적인 압축)
- **용도**: 메모리 극한 최적화
- **권장**: △ Q4_K_M 선호

Q5_K_M (5-bit K-quantization, Medium)

- **메모리 절감**: FP16 대비 ~40%
- **품질 저하**: 최소 (5% 미만 성능 손실)
- **속도**: 2.5-3.2배 빠름
- **용도**: 높은 품질이 필요한 애플리케이션
- **권장**: ✅ 품질 우선 시 선택

Q6_K (6-bit K-quantization)

- **메모리 절감**: FP16 대비 ~30%
- **품질 저하**: 매우 낮음 (3% 미만)
- **속도**: 2.2-2.8배 빠름
- **용도**: 거의 원본 품질 요구
- **권장**: ✅ 높은 품질 + 적당한 압축

Q8_0 (8-bit Quantization)

- **Full name**: 8-bit Linear Quantization (Type 0)
- **메모리 절감**: FP16 대비 ~20%
- **품질 저하**: 거의 없음 (2% 미만)
- **속도**: 2.0-2.5배 빠름
- **방법**: 기본 선형(균일) 양자화, 고급 기법 없음 (0)
- **용도**: 프로덕션 애플리케이션 (높은 품질 필수)
- **특징**: BF16 네이티브 모델도 품질 잘 유지
- **권장**: ✅ 연구 및 개발, 고품질 추론

6.3 양자화 비교 표

스킴	비트	메모리 (13B 모델)	품질 손실	속도 향상	사용 사례	권장도
FP16	16	26 GB	0%	1x	기준	✅ 연구
Q8_0	8	13 GB	<2%	2.0-2.5x	고품질 프로덕션	✅ 품질 우선
Q6_K	6	10.5 GB	❤️%	2.2-2.8x	거의 원본 품질	✅ 균형
Q5_K_M	5	9.1 GB	<5%	2.5-3.2x	좋은 품질	✅ 권장
Q4_K_M	4	7.9 GB	5-15%	3.0-4.0x	**최고 인기**	✅✅ 최우선
Q4_K_S	4	7.5 GB	10-20%	3.0-4.0x	공격적 압축	△
Q3_K_M	3	6.3 GB	15-30%	3.5-4.5x	제한된 하드웨어	△
Q2_K	2	5.4 GB	30%+	4.0-5.0x	실험적	❌

6.4 양자화 선택 가이드

연구 및 개발

- **권장**: FP16 또는 Q8_0
- **이유**: 최고 품질 유지, 정확한 평가
- **트레이드오프**: 높은 리소스 사용

프로덕션 배포 (클라우드)

- **권장**: Q5_K_M 또는 Q4_K_M
- **이유**: 품질과 비용 효율성 균형
- **트레이드오프**: 약간의 품질 감소, 큰 비용 절감

소비자 하드웨어 배포

- **권장**: Q4_K_M 또는 Q3_K_M
- **이유**: 제한된 하드웨어에서 실행 가능
- **트레이드오프**: 눈에 띄는 품질 감소, 넓은 접근성

모바일 및 엣지 애플리케이션

- **권장**: INT8 또는 Q4_K_S
- **이유**: 리소스 제약 환경 최적화
- **트레이드오프**: 품질 감소, 전력 및 메모리 효율성

6.5 양자화 파일명 예시

```
llama-3-8b-q4_k_m.gguf              # Llama 3 8B, Q4_K_M
mistral-7b-instruct-q5_k_m.gguf     # Mistral 7B, Q5_K_M
phi-3-mini-q8_0.gguf                # Phi-3 Mini, Q8_0
deepseek-r1-q4_k_m.gguf             # DeepSeek-R1, Q4_K_M
```

6.6 ComfyUI와 양자화

현재 상태 (2026년 2월):

- ComfyUI는 주로 **디퓨전 모델** (이미지/비디오 생성) 중심
- GGUF 양자화는 **LLM** (대규모 언어 모델)에 주로 사용
- ComfyUI에서 직접적인 GGUF 지원은 제한적

간접적 사용:

- LLM 기반 프롬프트 생성기를 외부에서 실행 (ollama, llama.cpp)
- 생성된 프롬프트를 ComfyUI로 전달

디퓨전 모델 양자화:

- FP16, BF16, FP8: 주로 텍스트 인코더 (T5) 양자화
- UNet/Transformer: 일반적으로 FP16 또는 BF16 유지

---

7. 주요 생성 모델 아키텍처

7.1 Stable Diffusion 시리즈

Stable Diffusion 1.x/2.x

- **아키텍처**: U-Net 기반 잠재 디퓨전 모델
- **파라미터**: 약 0.98B (SD 1.5)
- **해상도**: 512×512
- **텍스트 인코더**: CLIP-L (768차원)
- **VAE**: 8배 압축
- **특징**:
  - 가볍고 빠름
  - 소비자 GPU에서 실행 가능
- **한계**:
  - 낮은 해상도
  - 제한된 프롬프트 이해

Stable Diffusion XL (SDXL)

- **파라미터**: 약 3.5B
- **해상도**: 1024×1024 네이티브
- **텍스트 인코더**: CLIP-L + CLIP-G
- **VAE**: 개선된 SDXL VAE
- **특징**:
  - 더 나은 디테일과 품질
  - 두 개의 CLIP 모델 조합
  - 향상된 색상 정확도
- **VRAM 요구량**: ~8GB (FP16)

Stable Diffusion 3 (SD3)

- **아키텍처**: MMDiT (Multimodal Diffusion Transformer)
- **Full name**: Multimodal Diffusion Transformer
- **파라미터 범위**: 800M ~ 8B (여러 변형)
- **해상도**: 1024×1024 이상
- **텍스트 인코더**: CLIP-L + CLIP-G + T5-XXL
- **학습 방법**: Rectified Flow (RF)

주요 개선사항:

1. **MMDiT 아키텍처**:
   - 이미지와 언어에 **별도의 가중치 세트** 사용
   - 두 모달리티가 독립적으로 작동하면서 상호 참조
   - 트랜스포머 기반 (U-Net 대체)
2. **Rectified Flow (RF)**:
   - 데이터와 노이즈를 **선형 경로**로 연결
   - 확률적 샘플링 → 결정론적 벡터 필드
   - 더 적은 샘플링 스텝으로 고품질 생성
   - 재가중 RF: 경로 중간 부분에 더 많은 가중치
3. **유연한 텍스트 인코더**:
   - T5-XXL 제거 가능 (메모리 절감)
   - 시각적 미학 유지 (win rate 50%)
   - 타이포그래피 성능 약간 저하 (win rate 38%)

성능:

- 8B 모델: RTX 4090 (24GB VRAM)에서 34초 (1024×1024, 50 스텝)

- DALL·E 3, Midjourney v6, Ideogram v1 능가

장점:

- ✅ 최고 수준의 텍스트 이해
- ✅ 정확한 철자 (텍스트 렌더링)
- ✅ 복잡한 프롬프트 준수
- ✅ 확장성 (여러 크기 모델)

단점:

- ❌ 높은 VRAM 요구량
- ❌ 느린 생성 속도

7.2 Flux 시리즈

7.2.1 Flux.1

개요

- **개발**: Black Forest Labs (Stable Diffusion 원개발팀)
- **파라미터**: 12B (120억)
- **비교**: SDXL 3.5B의 약 3.4배
- **학습 방법**: Rectified Flow (RF)

아키텍처 특징

1. 하이브리드 아키텍처

- Multimodal Transformer Blocks: 이미지와 텍스트 처리
- Parallel Diffusion Transformer Blocks: 병렬 어텐션 레이어

2. 핵심 컴포넌트

Double-Stream Blocks:

- **두 개의 독립적인 스트림**: 잠재(이미지) 스트림, 프롬프트(텍스트) 스트림
- **계산 방식**: 순차적 (어텐션 → MLP)
- **가중치**: 각 스트림마다 별도
- **정규화**: Adaptive Layer Normalization (AdaLN)
- **특징**: SD3의 MMDiT 디자인과 유사

Single-Stream Blocks:

- **하나의 통합 스트림**: 이미지와 텍스트 결합
- **계산 방식**: 병렬 (어텐션 || MLP)

- **가중치**: 공유
- **특징**:
    - 경량화
    - 효율적 용량 확장
    - Flux 고유 혁신

3. Flow Matching

- **방식**: 결정론적 ODE (Ordinary Differential Equation)
- **샘플러**: FlowMatchEulerDiscreteScheduler
- **장점**:
    - 확률적 노이즈 스케줄 불필요
    - 더 빠르고 안정적인 생성
    - 복잡도 감소

4. Rotary Positional Embeddings (RoPE)

- **목적**: 위치 정보 인코딩
- **장점**: 더 나은 공간 관계 이해

5. 병렬 어텐션 레이어

- **동시 처리**: 여러 어텐션 헤드 병렬 실행
- **효율성**: 속도 향상

텍스트 인코더

- **구성**: CLIP-L (768차원) + T5-XXL (4096차원)
- **차이점**: CLIP-G 제외 (SD3 대비 간소화)

가이던스 메커니즘

- **Flux Dev**: 가이던스 임베딩 사용
- **Flux Schnell**: 가이던스 스케일 불필요 (0으로 설정)

Flux.1 변형

Flux.1 Pro

- **상용**: 최고 품질, API 접근
- **해상도**: 최대 2K
- **속도**: 느림

Flux.1 Dev

- **오픈소스**: 비상업적 사용

- **품질**: 높음
- **추론 스텝**: 20-50 스텝
- **VRAM**: ~16-20GB (FP16)

Flux.1 Schnell (빠름)

- **오픈소스**: 상업적 사용 가능
- **최적화**: 속도 우선
- **추론 스텝**: 3-6 스텝 (매우 빠름)
- **VRAM**: ~12-16GB (FP16)
- **품질**: Dev보다 약간 낮음

Flux.1.1 Pro Ultra (2026)

- **해상도**: 최대 4 메가픽셀
- **특징**: 초고해상도 생성

장점

- ✅ 매우 높은 품질
- ✅ 정확한 프롬프트 준수
- ✅ 텍스트 렌더링 우수
- ✅ 자연스러운 이미지
- ✅ 복잡한 구도 처리

단점

- ❌ 높은 VRAM 요구량
- ❌ 느린 생성 속도 (Dev, Pro)
- ❌ 큰 모델 크기

7.2.2 Flux.2

개요

- **개발**: Black Forest Labs
- **출시**: 2025년 11월
- **파라미터**: 32B (320억)
- **Full name**: Flux 2.0 Latent Flow Matching Transformer
- **비교**: Flux.1 (12B)의 약 2.67배 규모

핵심 혁신: 완전히 새로운 아키텍처

1. Mistral-3 24B Vision-Language Model 통합

- **Full name**: Mistral-Small-3.2-24B-Instruct-2506
- **역할**: 텍스트와 이미지를 이해하는 인코더
- **장점**:
  - 실세계 지식 (world knowledge) 이해
  - 공간 관계 파악
  - 물리적 속성 이해
  - 문맥적 의미 파악
- **결과**: "환각(hallucination)" 문제 극적 감소

2. Latent Flow Matching Architecture

기존 디퓨전 vs Flow Matching:

```
기존 디퓨전 모델:
노이즈 → 반복적 디노이징 → 이미지 (많은 스텝 필요)

Flux 2 Flow Matching:
텍스트 설명 → 직접 매핑 → 잠재 공간 → 이미지 (적은 스텝)
```

장점:

- ✅ **빠른 수렴**: 고품질 출력을 위한 샘플링 스텝 감소
- ✅ **향상된 프롬프트 준수**: 복잡한 지시사항 정확한 해석
- ✅ **일관성 개선**: 시드 제어로 재현 가능한 결과
- ✅ **높은 충실도**: 해상도 전반에 걸친 디테일 보존

3. 혁신적 잠재 공간 (Latent Space)

- **재학습**: 잠재 공간을 처음부터 다시 학습
- **목표**: "Learnability-Quality-Compression" 트릴레마 해결
- **결과**:
  - 더 나은 학습 가능성 (learnability)
  - 더 높은 이미지 품질
  - 더 나은 압축 효율성

4. MM-DiT + Parallel DiT 개선

구조 변화:

- **Double-Stream Blocks**: 8개 (Flux.1: 19개)
- **Single-Stream Blocks**: 48개 (Flux.1: 38개)
- **파라미터 분포**:
  - Flux.1: ~54% double-stream, ~46% single-stream
  - Flux.2: ~24% double-stream, ~73% single-stream

최적화:

- 시간 및 가이던스 정보를 모든 블록에서 공유
- 편향(bias) 파라미터 제거 → 경량화
- QKV 프로젝션과 FF 입력 프로젝션 융합 → 완전 병렬 트랜스포머

5. 단일 텍스트 인코더

- **Flux.1**: CLIP-L + T5-XXL (2개)
- **Flux.2**: Mistral Small 3.1 (1개)
- **장점**:
  - 프롬프트 임베딩 계산 크게 단순화
  - 최대 시퀀스 길이: 512 토큰
  - 더 나은 언어 이해

기술 사양

```
파라미터: 32B (320억)
최대 해상도: 4 메가픽셀 (4MP)
텍스트 입력 토큰: 최대 32,000
생성 속도: 10초 미만 (고사양 GPU)
종횡비: 모든 비율 지원
제어 시스템: JSON 기반
```

Flux.2 변형

Flux.2 [Pro]

- **상용**: 최고 품질 상업용 엔드포인트
- **최적화**: 최대 충실도 및 제어 필요한 프로덕션 워크플로우
- **접근**: BFL API, BFL Playground
- **가격**: 유료

Flux.2 [Flex]

- **가변 스텝**: 6~50 스텝 사이 조절 가능
- **트레이드오프**: 디테일과 속도 균형
- **용도**: 사용 사례에 따라 품질-속도 조절
- **접근**: BFL API

Flux.2 [Max]

- **최고 품질**: 절대적 품질 우선
- **용도**: 프로페셔널 작업
- **접근**: BFL Playground, API

- **가격**: 유료 (프리미엄)

Flux.2 [Dev]

- **오픈 웨이트**: 비상업적 사용 가능
- **효율성**: 비용 효과적
- **용도**:
  - 개발자 파이프라인 테스트
  - 대량 컨셉 생성
  - 레이아웃 테스트
- **비용**: 3 크레딧/이미지 (Scenario 기준)
- **특수 기능**: "Go Fast" 토글 (최적화된 속도)
- VRAM: ~20-24GB (FP16)

주요 개선사항

1. 포토리얼리즘

- 물리적으로 그럴듯한 이미지
- 정확한 조명 및 그림자
- 현실적인 재질 속성
- 일관된 공간 논리

2. 텍스트 렌더링

- 이미지 내 정확한 철자
- 복잡한 타이포그래피 처리
- 다중 줄 텍스트 레이아웃

3. 프롬프트 준수

- 복잡하고 구조화된 지시사항 신뢰성 있게 실행
- 다중 파트 프롬프트 처리
- 구성적 제약 존중
- 지정된 레이아웃 유지

4. World Knowledge

- 현실적 물리 및 조명 동작
- 일관된 공간 논리
- 예상되는 재질 속성
- 자연스러운 환경 조명

5. 다중 참조 제어

- **최대 10개 이미지** 참조 입력

- 정확한 색상 매칭
- 4MP 해상도 편집
- JSON 기반 제어 시스템

성능 벤치마크

- **LM Arena 순위**: 최상위권
- **프롬프트 준수**: DALL·E 3, Midjourney v6 능가
- **텍스트 렌더링**: 업계 최고 수준
- **생성 속도**: RTX 4090에서 10초 미만 (1024×1024)

장점

- ✅ 최고 수준의 포토리얼리즘
- ✅ 뛰어난 프롬프트 이해
- ✅ 정확한 텍스트 렌더링
- ✅ World knowledge 기반 생성
- ✅ 다중 참조 제어
- ✅ LoRA 파인튜닝 지원 우수
- ✅ 프롬프트 엔지니어링 복잡도 감소

단점

- ❌ 매우 높은 VRAM 요구량 (20-24GB)
- ❌ 대용량 모델 크기
- ❌ Flux.1보다 느린 생성 속도 (Dev 기준)
- ❌ Pro/Max 버전은 유료

Flux.1 vs Flux.2 비교

특성	Flux.1	Flux.2
파라미터	12B	32B
아키텍처	디퓨전 기반	Flow Matching
텍스트 인코더	CLIP-L + T5-XXL (2개)	Mistral Small 3.1 (1개)
Double-Stream	19 블록	8 블록
Single-Stream	38 블록	48 블록
최대 해상도	1024×1024+	4MP (4 메가픽셀)
텍스트 토큰	512	32,000
참조 이미지	제한적	최대 10개

특성	Flux.1	Flux.2
World Knowledge	제한적	Mistral-3 24B 통합
VRAM (Dev)	16-20GB	20-24GB

ComfyUI에서 Flux.2 사용

필수 노드:

```
Load Checkpoint (Flux.2 Dev)
    ↓
CLIP Text Encode (Mistral Small 3.1)
    ↓
KSampler (FlowMatchEulerDiscrete)
    ↓
VAE Decode
    ↓
Save Image
```

권장 설정:

- **샘플러**: FlowMatchEulerDiscrete
- **스텝**: 10-30 (Flux.1보다 적어도 됨)
- **CFG Scale**: 0 또는 낮은 값 (모델에 따라)
- **해상도**: 1024×1024 ~ 2048×2048

7.3 Qwen Image

개요

- **Full name**: Qwen Image Generation Foundation Model
- **개발**: Alibaba (알리바바)
- **출시**: 2026년 2월
- **파라미터**: 20B (200억) - Dense 모델
- **아키텍처**: MMDiT (Multimodal Diffusion Transformer)

핵심 혁신

1. 복잡한 텍스트 렌더링의 혁명

Qwen Image의 차별점:

- 텍스트를 단순한 픽셀 집합이 아닌 **전문적 처리 대상**으로 취급

- 이중 인코딩 (dual encoding)
- 점진적 학습 (progressive training)
- 아키텍처 최적화

성과:

- 문자 오해석 감소
- 문맥적 정확성 보장
- 비교 불가능한 정확도

2. 다국어 텍스트 지원 (세계 최고)

지원 언어:

- **알파벳 언어**: 영어, 스페인어, 프랑스어, 독일어 등
- **표의문자 언어**: 중국어 (간체/번체), 일본어, 한국어 등

텍스트 렌더링 능력:

```
단일 행 텍스트 ✅
다중 행 레이아웃 ✅
단락 수준 의미론 ✅
세밀한 시각적 디테일 ✅
복잡한 타이포그래피 ✅
```

예시 사용 사례:

- 영화 포스터 제목 + 다중 행 자막
- PPT 슬라이드 정확한 레이아웃
- 책 표지 디자인
- 인포그래픽 제작
- 광고 배너 생성

3. 포괄적 데이터 엔지니어링

훈련 데이터셋:

- LongText-Bench: 긴 텍스트 처리
- ChineseWord: 중국어 텍스트 전문
- TextCraft: 텍스트 렌더링 특화
- 다양한 스타일 및 레이아웃 데이터

학습 전략:

- Progressive learning (점진적 학습)

- Multi-task training (다중 작업 학습)
- Scalable infrastructure optimization (확장 가능 인프라)

주요 기능

1. 이미지 생성

능력:

- 복잡한 텍스트 프롬프트로 고품질 이미지 생성
- 스타일적으로 다양한 출력
- 포토리얼리스틱부터 예술적 스타일까지
- 애니메이션, 인포그래픽, 예술 작품 등

강점:

- 복잡한 언어 구조 깊은 이해
- 시각적으로 매력적인 출력
- 의미적으로 정확한 결과

2. 이미지 편집 (Context-Aware)

편집 기능:

- 스타일 전환 (Style Transfer)
- 텍스트 편집 (Text Editing)
- 배경 교체 (Background Replacement)
- 객체 추가 (Object Addition)
- 객체 제거 (Object Removal)
- 객체 대체 (Object Substitution)
- 포즈 조작 (Pose Manipulation)

특징:

- 의미 무결성 (semantic integrity) 보존
- 시각적 사실성 (visual realism) 유지
- 일관된 편집 결과

3. 이미지 이해 및 분석

분석 작업:

- **객체 검출**: 바운딩 박스 생성
    - 예: "지하철 장면에서 허스키 개 찾기"
- **의미론적 분할** (Semantic Segmentation)
- **깊이 추정** (Depth Estimation)

- **엣지 검출** (Canny Edge Detection)
- **초해상도** (Super-Resolution)
- **새로운 시점 합성** (Novel View Synthesis)

응용 분야:

- 자동화
- 전자상거래 태깅
- 시각적 분석
- 콘텐츠 이해

4. 이미지 대 이미지 (Image-to-Image)

기능:

- 참조 이미지에서 피사체 또는 윤곽 특징 추출
- 2차 편집으로 유연한 요구 충족
- 다양한 변형 생성

5. 실시간 드로잉 캔버스

혁신적 워크플로우:

- 스케치하거나 파라미터 조정 시 **동시에** 미리보기 생성
- 전통적 "그리기-대기-수정" 워크플로우 탈피
- 다중 이미지 융합 지원
- AI가 원근 및 조명 자동 조정

6. 다중 이미지 융합

- 여러 스케치를 동일 캔버스에 오버레이
- AI가 원근감 통합
- 조명 일관성 자동 조절

기술 아키텍처

Multimodal Diffusion Transformer (MMDiT)

구성:

```
텍스트 입력 + 이미지 입력
     ↓
MMDiT (디퓨전 + 트랜스포머 융합)
     ↓
점진적 개선 (Progressive Refinement)
```

```
      ↓
고품질 이미지 출력
```

특징:

- 텍스트와 이미지 입력을 효율적으로 처리
- 디퓨전을 통한 점진적 디노이징
- 트랜스포머 논리로 요소 간 복잡한 관계 처리

효율성 최적화

- **최소 VRAM**: 4GB (FP8 양자화 + 오프로딩 사용)
- **소비자 하드웨어**: 일반 GPU에서 실행 가능
- **실용적 설계**: 실세계 사용 최적화

벤치마크 성능

이미지 생성:

- GenEval: 최고 수준
- DPG-Bench: 최고 수준
- OneIG-Bench: 최고 수준

이미지 편집:

- GEdit: 최고 수준
- ImgEdit: 최고 수준
- GSO: 최고 수준

텍스트 렌더링:

- LongText-Bench: 기존 모델 능가
- ChineseWord: 기존 모델 능가
- TextCraft: 기존 모델 능가

사용 워크플로우

기본 생성:

```
1. 입력 프롬프트
   - 영어 또는 중국어로 상세한 설명
   - 텍스트 내용 포함 가능

2. 종횡비 선택
   - 정사각형, 세로, 가로, 커스텀
```

```
3. 생성 및 대기
   - Qwen-Image 20B MMDiT 모델 실행
   - 네이티브 텍스트 렌더링으로 고품질 이미지 생성
```

장점

- ✅ 세계 최고 수준의 텍스트 렌더링 (특히 다국어)
- ✅ 중국어/한글 등 표의문자 완벽 지원
- ✅ 포괄적 이미지 편집 기능
- ✅ 이미지 이해 및 분석 능력
- ✅ 실시간 드로잉 캔버스
- ✅ 낮은 VRAM 요구사항 (4GB부터 가능)
- ✅ 오픈소스 (Hugging Face, GitHub, ModelScope)
- ✅ 다양한 스타일 지원

단점

- ❌ 상대적으로 최근 출시 (2026년 2월)
- ❌ 영어권 이외 지역에서 더 강점 (서양 스타일은 다른 모델과 비슷)
- ❌ ComfyUI 네이티브 지원 아직 제한적 (커스텀 노드 필요 가능)

Qwen Image 사용 사례

1. 다국어 포스터 제작

```
프롬프트: "영화 포스터, 제목 '想象力释放' (중국어),
           부제 'Imagination Unleashed' (영어),
           SF 스타일, 네온 조명"
→ 정확한 다국어 텍스트 렌더링
```

2. 인포그래픽 생성

```
프롬프트: "데이터 시각화 인포그래픽, 제목 '2026 AI 트렌드',
           한글 레이블, 차트 포함, 전문적 디자인"
→ 정확한 한글 텍스트 + 레이아웃
```

3. 광고 배너

```
프롬프트: "온라인 쇼핑 배너, 프로모션 텍스트 '50% 할인',
           제품 이미지, 현대적 디자인"
→ 정확한 텍스트 + 시각적 매력
```

비교: Qwen Image vs 다른 모델

특성	Qwen Image	SDXL	Flux.2	Midjourney
파라미터	20B	3.5B	32B	비공개
다국어 텍스트	✅✅ 최고	❌ 약함	✅ 좋음	❌ 약함
중국어/한글	✅✅ 완벽	❌ 불가	△ 제한적	❌ 불가
텍스트 렌더링	✅✅ 최고	❌ 약함	✅✅ 우수	✅ 좋음
이미지 편집	✅✅ 포괄적	✅ 기본	✅ 우수	✅ 제한적
이미지 분석	✅✅ 포함	❌ 없음	❌ 없음	❌ 없음
오픈소스	✅	✅	△ Dev만	❌
최소 VRAM	4GB	8GB	20GB	N/A (클라우드)

7.4 비디오 생성 모델 (2026)

7.4.1 Wan2.2

- **Full name**: Wan Video Generation Model 2.2
- **개발**: 오픈소스 커뮤니티
- **출시**: 2025년
- **지위**: **최고의 오픈소스 비디오 생성 모델**
- **아키텍처**: Mixture-of-Experts (MoE) 디퓨전 모델
    - **High-Noise Expert**: 초기 단계, 전체 레이아웃 집중
    - **Low-Noise Expert**: 후기 단계, 비디오 디테일 정제
- **파라미터**:
    - T2V-1.3B: 1.3B 파라미터
    - A14B: 14B 파라미터
- **학습 데이터**:
    - 15억 개 비디오
    - 100억 개 이미지
- **성능**: VBench 점수 84.7%+
- **특징**:
    - 복잡한 다이내믹스 처리
    - 공간 관계 이해
    - 다중 객체 상호작용
    - **중국어 및 영어 텍스트 생성** (최초)
- **VRAM 요구량**:

    - 1.3B 모델: 8.19 GB (거의 모든 소비자 GPU 호환)
- **생성 시간**: RTX 4090에서 5초 480P 비디오 약 4분

가격: 완전 무료, 오픈소스 (GitHub, Hugging Face)

지원 모드:

- Text-to-Video (1.3B, 14B)
- 해상도: 480P, 720P

7.4.2 LTX-2

- **개발**: Lightricks
- **출시**: 2026년 1월
- **지위**: 최초 오픈소스 4K 오디오+비디오 통합 모델
- **파라미터**: 19B (비디오 14B, 오디오 5B)
- **특징**:
    - 네이티브 4K 출력
    - 동기화된 표현적 사운드
    - 로컬 배포 가능

7.4.3 HunyuanVideo 1.5

- **개발**: Tencent (텐센트)
- **특징**:
    - 오픈소스
    - 소비자 하드웨어 최적화
    - 완전한 커스터마이징 제어

7.4.4 Kling 2.6

- **개발**: Kuaishou (중국 콰이쇼우)
- **출시**: 2025년 12월
- **핵심 혁신**: **동시 오디오-비주얼 생성**
- **아키텍처**:
    - 디퓨전 기반 트랜스포머 (DiT)
    - 3D Variational Autoencoder (VAE) 네트워크
- **해상도**: 최대 1080p
- **프레임레이트**: 30fps
- **길이**: 최대 2분
- **특징**:
    - 자연스러운 보이스오버
    - 음향 효과

    - 주변 분위기 사운드
    - 단일 패스 생성 (비디오+오디오 동시)
- **성과**: 1000만+ 비디오 생성

가격:

- 일반 모드: 1,000 크레딧 (오디오 없음), 1,500 크레딧 (오디오 포함)
- 720p/1080p 동일 가격

7.4.5 Veo 3.1

- **개발**: Google DeepMind
- **출시**: 2025년
- **특징**:
    - 네이티브 오디오 생성
    - 고품질 물리 시뮬레이션
    - 신속한 반복 생성

가격:

- 일반 모드: 2,000 크레딧 (오디오 없음), 2,500 크레딧 (오디오 포함)

7.4.6 Sora 2

- **개발**: OpenAI
- **특징**:
    - 시네마틱 품질
    - 동기화된 오디오
    - 정교한 물리 시뮬레이션

7.5 오디오 생성 모델

생성형 AI는 이미지와 비디오를 넘어 오디오 영역으로 확장되고 있습니다. 2026년 현재, 텍스트 프롬프트로 음악, 음향 효과, 보이스를 생성할 수 있는 모델들이 등장했습니다.

7.5.1 ACE-Step v1.0

개요

- **Full name**: A Step Towards Music Generation Foundation Model
- **개발**: ACE Studio + StepFun (협력 개발)
- **출시**: 2025년 초
- **라이선스**: Apache 2.0 (오픈소스)

- **파라미터**: 3.5B (35억)
- **목적**: AI 음악 생성의 속도, 일관성, 제어 가능성 한계 극복

핵심 아키텍처

통합 아키텍처 (Holistic Architecture)

```
디퓨전 기반 생성
    +
Deep Compression AutoEncoder (DCAE) from Sana
    +
경량 Linear Transformer
```

구성 요소:

- **디퓨전 모델**: 음악 생성 기반
- DCAE (Deep Compression AutoEncoder):
    - Sana 프로젝트에서 가져온 압축 인코더
    - 미세한 음향 디테일 보존
    - 고압축률 달성
- Linear Transformer:
    - 경량 설계
    - 장거리 구조적 일관성 유지

REPA (Semantic Representation Alignment)

- **Full name**: Representation Alignment using MERT and m-hubert
- **목적**: 의미 표현 정렬
- **방법**:
    - **MERT**: Music Understanding Model
    - **m-hubert**: Multilingual HuBERT
    - 학습 중 의미 표현 정렬
- **효과**:
    - 빠른 수렴
    - 텍스트-음악 정렬 개선
    - 출력 일관성 향상

주요 특징

1. 초고속 생성

- **A100 GPU**: 4분 음악을 20초에 생성
- **RTX 4090**: Real-Time Factor (RTF) 34.48x
- **RTX 3090**: Real-Time Factor (RTF) 12.76x

- **비교**: LLM 기반 모델 대비 **15배 빠름**

2. **긴 음악 생성**

- **최대 길이**: 4분
- **품질**: 전체 길이에 걸쳐 일관성 유지
- **구조**: 멜로디, 하모니, 리듬 메트릭에서 우수

3. **다국어 지원**

- **지원 언어**: 19개 언어
- 텍스트 프롬프트 다국어 입력
- 가사 다국어 생성

4. **음악 제어** 입력 방식:

- 태그 (tags)
- 장르 (genres)
- 장면 설명 (scene descriptions)
- 가사 (lyrics)

5. **고급 제어 기능**

Variations Generation (**변형 생성**)

- **방법**:
  - Flow-matching 모델로 초기 노이즈 생성
  - trigFlow 공식으로 추가 가우시안 노이즈 추가
- **제어**:
  - 노이즈 혼합 비율 조절
  - 변형 정도 제어 가능

Repainting (**재페인팅**)

- **목적**: 특정 오디오 섹션 재생성
- **방법**:
  - 노이즈 추가
  - ODE 프로세스 중 마스크 제약 적용
- **용도**:
  - 국지적 편집
  - 섹션 내 변형

Lyric Editing (**가사 편집**)

- **기술**: Flow-edit 기술
- **특징**:

    - 국지적 가사 수정
    - 원본 멜로디 보존
    - 보컬 음색 보존
    - 반주 보존

음악적 특징

1. 악기 스타일

- 다양한 장르 및 스타일의 악기 음악 생성
- 각 악기에 적절한 음색 및 표현력
- 복잡한 편곡 가능 (다중 악기)
- 음악적 일관성 유지

2. 보컬 기법

- 다양한 보컬 스타일 및 기법
- 여러 보컬 표현 지원
- 다양한 창법 및 스타일

성능 메트릭

- **음악 일관성**: 최고 수준
- **가사 정렬**: LLM 기반 모델 능가
- **멜로디 품질**: 우수
- **하모니**: 우수
- **리듬**: 우수

지원 기능

- Lyric2Vocal: 가사에서 보컬 생성
- Text2Samples: 악기 샘플 생성
- Music Remixing: 음악 리믹스
- Style Transfer: 스타일 전환
- Audio Extension: 기존 오디오 확장
- Singing2Accompaniment: (예정) 노래에서 반주 생성

7.5.2 ACE-Step v1.5 (2026년 최신)

개요

- **출시**: 2026년 1월
- **파라미터**: 미공개 (v1.0 개선판)
- **목표**: 상업급 품질을 소비자 하드웨어로

획기적 개선사항

1. 상업급 품질

- **평가**: 음악 일관성 점수 **4.72**
- **비교**: 대부분의 상업 음악 모델을 능가
- **품질**: 전문가 수준

2. 극한의 속도

- **RTX 5090**: 4분 곡을 **~1초** 생성
- **RTX 3090**: 4분 곡을 **10초 이하** 생성
- **A100**: 4분 곡을 **2초 이하** 생성
- **이전 버전 대비**: 10-20배 빠름

3. 소비자 하드웨어 최적화

- **최소 VRAM**: 4GB 미만
- **로컬 실행**: 가능
- **접근성**: 대중화

4. 50+ 언어 지원

- **강력한 지원**: 영어, 중국어, 일본어, 한국어, 스페인어, 독일어, 프랑스어, 포르투갈어, 이탈리아어, 러시아어 등
- **총 지원**: 50개 이상 언어
- **프롬프트 준수**: 엄격한 프롬프트 준수

5. LoRA 파인튜닝

- **지원**: ComfyUI에서 경량 개인화
- **학습**: 몇 곡만으로 스타일 캡처 가능
- **용도**: 개인 스타일 학습

혁신적 아키텍처

Hybrid Reasoning-Diffusion Architecture (하이브리드 추론-디퓨전 아키텍처)

1. Language Model as Composer Agent (작곡가 에이전트로서의 언어 모델)

- **역할**: 고수준 플래너 (high-level planner)
- **기능**:
    - 모호한 사용자 프롬프트를 정확한 구조 청사진으로 재구성
    - 가사 포매팅
    - 지속시간 계산
    - 쿼리 확장
- Chain-of-Thought Planning:

    - 메타데이터, 가사, 캡션을 Chain-of-Thought 추론으로 합성
    - 디퓨전 프로세스 가이드
    - 더 일관된 장편 작곡

2. Diffusion Transformer as Acoustic Renderer (음향 렌더러로서의 디퓨전 트랜스포머)

- **역할**: 전문 음향 렌더러
- **집중**: 음향 풍부함 및 악기 분리
- **자유**: 의미적 모호성에서 해방

3. 증류 알고리즘 (Distillation Algorithm)

- **압축**: 50 스텝 → 4-8 스텝
- **속도**: 100배 이상 빨라짐
- **품질**: 신호 대 잡음비 성능 향상

4. **통합 Omni-Task 프레임워크** 단순 텍스트-음악 합성을 넘어:

- Cover Generation (커버 생성)
- Seamless Repainting (매끄러운 재페인팅)
- Track Extraction (트랙 추출)
- Vocal-to-BGM Conversion (보컬-배경음악 변환)

5. 순수 파형 도메인 1D VAE

- **이전**: 2D Mel-spectrogram (위상 손실, 고주파 해상도 부족)
- **현재**: 순수 파형 도메인 1D VAE
- **압축**: 48kHz 스테레오를 64차원 잠재 공간 (25Hz)으로 압축
- **압축률**: 1920배
- **품질**: 거의 무손실 지각 품질

6. Intrinsic Reinforcement Learning (내재적 강화 학습)

- **방법**: 모델 내부 메커니즘만 사용
- **장점**: 외부 보상 모델 또는 인간 선호도 편향 제거
- **결과**: 순수한 정렬

확장성

- **짧은 루프**: 지원
- **긴 작곡**: 최대 10분까지 확장 가능

성능 비교

모델	4분 곡 생성 시간 (A100)	4분 곡 생성 시간 (RTX 3090)	최소 VRAM	품질 점수
ACE-Step v1.0	20초	~50초	~8GB	4.5
ACE-Step v1.5	<2초	<10초	<4GB	4.72
Suno v3	~60초 (API)	N/A (클라우드)	N/A	4.6
Udio	~50초 (API)	N/A (클라우드)	N/A	4.5
LLM-based (예: Yue)	300초+	N/A	높음	4.3

ComfyUI 통합 (v1.5)

- **네이티브 워크플로우**: ComfyUI에서 직접 실행 가능
- **노드 지원**:
  - ACE-Step 로더
  - 텍스트 프롬프트 입력
  - 가사 입력
  - 변형 생성
  - 재페인팅
  - 가사 편집
- **LoRA 파인튜닝**: ComfyUI에서 경량 학습 가능

사용 예시

기본 음악 생성:

```
프롬프트: "Upbeat electronic dance music, 128 BPM, energetic synths,
          driving bass, festival vibes"
→ 4분 EDM 트랙 생성 (1-2초)
```

가사 포함 생성:

```
가사: "Walking down the street, feeling the beat..."
장르: "Pop rock"
→ 가사 정렬된 팝록 곡
```

커버 생성:

```
원본 곡 업로드 + 스타일: "Jazz arrangement"
→ 재즈 스타일 커버 생성
```

장점

- ✅ 상업급 품질 (4.72 점수)
- ✅ 극한의 속도 (1-10초, 4분 곡)
- ✅ 소비자 하드웨어 (<4GB VRAM)
- ✅ 50+ 언어 지원
- ✅ LoRA 파인튜닝
- ✅ 오픈소스 (Apache 2.0)
- ✅ 다목적 (합성, 편집, 커버, 추출)
- ✅ ComfyUI 네이티브 지원

단점

- ❌ 상대적으로 최신 (2026년 1월)
- ❌ 복잡한 오케스트라 편곡은 제한적
- ❌ 보컬 품질은 전문 보컬 모델보다 약간 낮음

7.5.3 Stable Audio 2.0

개요

- **개발**: Stability AI
- **출시**: 2024년 4월
- **해상도**: 44.1kHz 스테레오
- **최대 길이**: 3분 10초
- **구조**: 인트로, 전개, 아웃트로 포함

핵심 기능

1. 전체 길이 트랙 생성

- **길이**: 최대 3분 10초
- **구조**:
  - Intro (인트로)
  - Development (전개)
  - Outro (아웃트로)
- **품질**: 일관된 음악 구조
- **출력**: 스테레오 사운드 이펙트

2. Audio-to-Audio 생성

- **업로드**: 오디오 샘플 업로드
- **변환**: 자연어 프롬프트로 변환
- **제약**: 저작권 없는 자료만 (고급 콘텐츠 인식)

- **용도**: 아이디어를 완전히 제작된 샘플로

3. Variations and Sound Effects

- **사운드 이펙트**: 키보드 타이핑, 군중 함성, 도시 소음 등
- **변형**: 다양한 버전 생성
- **품질**: 전문가 수준

4. Style Transfer (스타일 전환)

- **기능**: 생성되거나 업로드된 오디오의 스타일 수정
- **조정**: 프로젝트 특정 스타일 및 톤에 맞춤
- **통합**: 생성 프로세스 내 원활한 수정

아키텍처

1. 고압축 Autoencoder

- **역할**: 원본 오디오 파형을 훨씬 짧은 표현으로 압축
- **목적**: 장시간 규모 성능 개선
- **결과**: 더 효율적인 처리

2. Diffusion Transformer (DiT)

- **유사**: Stable Diffusion 3에 사용된 DiT
- **이전**: U-Net 대체
- **장점**:
    - 긴 시퀀스 데이터 조작에 더 능숙
    - 대규모 구조 인식 및 재현
    - 고품질 음악 작곡에 필수적인 구조

조합 효과:

- 전체 트랙의 일관된 구조 생성
- 장시간 규모의 음악적 논리 유지

학습 데이터

- **독점 라이선스**: AudioSparx 음악 라이브러리
- **옵트아웃**: 크리에이터 옵트아웃 요청 존중
- **보상**: 공정한 크리에이터 보상 보장

접근

- **웹사이트**: stableaudio.com (무료 체험)
- **API**: Stable Audio API

- **가격**: 프리미엄 및 무료 티어

배경

- **Stable Audio 1.0** (2023년 9월): 첫 번째 상업적 실행 가능 AI 음악 도구
- **수상**: TIME's Best Inventions of 2023

7.5.4 Stable Audio Open

개요

- **출시**: 2024년 6월
- **라이선스**: 오픈소스 (비상업적 연구용)
- **최적화**: 짧은 오디오 샘플, 사운드 이펙트, 제작 요소
- **최대 길이**: 47초

목적

- 사운드 디자이너, 음악가, 개발자를 위한 오픈 모델
- 드럼 비트, 악기 리프, 앰비언트 사운드, 폴리 녹음 생성
- 책임 있는 개발 우선

아키텍처

3가지 구성 요소:

1. **Autoencoder**: 파형 압축
2. **T5 기반 텍스트 임베딩**: 조건화
3. **Transformer 기반 Diffusion Model**: Autoencoder의 잠재 공간에서 작동

학습 데이터

- **FreeSound**: 472,618 오디오 녹음
- **Free Music Archive**: 13,874 오디오
- **총**: 486,492 오디오 녹음
- **라이선스**: CC0, CC BY, CC Sampling+ (저작권 음악 제외)

특징

- **해상도**: 44.1kHz 스테레오
- **파인튜닝**: 커스텀 오디오 데이터로 파인튜닝 가능
    - 예: 드러머가 자신의 드럼 녹음으로 파인튜닝
- **도구**: stable-audio-tools 라이브러리
- **모델 가중치**: Hugging Face에서 다운로드 가능

용도

- 드럼 루프
- 폴리 (Foley) 사운드
- 악기 리프
- 앰비언트 텍스처
- 짧은 음악 클립

제한사항

- ❌ 전체 곡 생성 최적화되지 않음
- ❌ 멜로디 또는 보컬 최적화되지 않음
- ❌ 짧은 샘플 전문

7.5.5 Stable Audio Open Small (2025년 11월)

개요

- **개발**: Stability AI + Arm (협력)
- **출시**: 2025년 11월
- **파라미터**: 341M (3억 4100만)
- **비교**: Stable Audio Open (1.1B)의 약 1/3
- **최적화**: Arm CPU에서 실행

획기적 특징

1. 온디바이스 실행 (스마트폰)

- **플랫폼**: Arm CPU (전 세계 스마트폰 99%)
- **속도**: 스마트폰에서 11초 오디오를 8초 이하에 생성
- **오프라인**: 클라우드 없이 작동
- **KleidiAI**: Arm의 KleidiAI 라이브러리로 최적화

2. 최고 속도

- **주장**: 시장에서 가장 빠른 스테레오 텍스트-오디오 모델
- **생성 속도**: 실시간보다 빠름
- **파인튜닝**: 더 빠른 파인튜닝

3. 경량 및 효율성

- **Arm CPU**: 완전히 Arm CPU에서 실행
- **하드웨어 요구사항**: 최소
- **전력 효율**: 낮은 전력 소비

- **비용 절감**: 계산 시간 비용 감소

4. 엣지 최적화

- **KleidiAI**: Arm의 라이브러리로 엣지 효율성 극대화
- **모바일 워크플로우**: 실시간 생성 및 응답성
- **배포**: Arm 기반 스마트폰 및 엣지 디바이스에 완벽

용도

- 드럼 루프
- 폴리 사운드
- 악기 리프
- 앰비언트 텍스처
- 짧은 사운드 이펙트

접근

- **라이선스**: Stability AI Community License (상업/비상업 무료)
- **모델**: Hugging Face
- **코드**: GitHub
- **논문**: arXiv
- **학습 경로**: Arm Learning Path

제한사항

- **언어**: 영어만 지원
- **보컬**: 현실적인 보컬 생성 불가
- **고품질 곡**: 고품질 전체 곡 생성 불가
- **음악 스타일**: 서양 편향 (학습 데이터 특성)

7.5.6 오디오 생성 모델 비교표

모델	파라미터	최대 길이	해상도	속도 (4분 기준)	주요 강점	라이선스
ACE-Step v1.0	3.5B	4분	48kHz	20초 (A100)	균형잡힌 품질과 속도	Apache 2.0
ACE-Step v1.5	미공개	10분	48kHz	1-2초 (A100)	최고 속도 + 상업급 품질	Apache 2.0
Stable Audio 2.0	미공개	3분 10초	44.1kHz	~60초 (API)	전체 트랙 구조	상업 (유료)
Stable Audio Open	미공개	47초	44.1kHz	빠름	오픈소스, 파인튜닝	비상업

모델	파라미터	최대 길이	해상도	속도 (4분 기준)	주요 강점	라이선스
Stable Audio Open Small	341M	11초	44.1kHz	8초 (스마트폰)	온디바이스, 초경량	상업/비상업

7.5.7 ComfyUI에서 오디오 생성 사용

기본 워크플로우 (ACE-Step):

```
Text Prompt Input
    ↓
ACE-Step Loader
    ↓
Generate Audio
    ↓
Save Audio (WAV/MP3)
```

고급 워크플로우 (편집 포함):

```
Load Audio (선택사항)
    ↓
Text Prompt + Lyrics
    ↓
ACE-Step Generator
    ↓
Repainting / Lyric Editing (선택사항)
    ↓
Save Audio
```

Stable Audio 통합:

- 현재 주로 웹 인터페이스 및 API
- ComfyUI 커스텀 노드 가능 (커뮤니티 개발 중)

7.5.8 실습 프로젝트 아이디어 (오디오)

초급:

1. **기본 음악 생성**: 간단한 프롬프트로 짧은 곡 생성
2. **사운드 이펙트**: 특정 사운드 이펙트 생성 (예: 발소리, 문 닫는 소리)
3. **장르 실험**: 다양한 장르 프롬프트 테스트

중급:

1. **가사 포함 곡**: 커스텀 가사로 완전한 곡 생성
2. **스타일 전환**: 기존 음악을 다른 스타일로 변환
3. **루프 생성**: 반복 가능한 드럼/베이스 루프

고급:

1. **LoRA 파인튜닝**: 자신의 음악 스타일 학습
2. **커버 생성**: 기존 곡의 AI 커버 버전
3. **다중 트랙 작곡**: 여러 악기 트랙 생성 및 믹싱

7.6 모델 비교 표

모델	파라미터	해상도	텍스트 인코더	아키텍처	특징	VRAM (FP16)
SD 1.5	0.98B	512^2	CLIP-L	U-Net	가볍고 빠름	~4GB
SDXL	3.5B	1024^2	CLIP-L+G	U-Net	더 나은 품질	~8GB
SD3	0.8-8B	1024^2+	CLIP-L+G+T5	MMDiT	최고 텍스트 이해	~12-24GB
Flux.1 Dev	12B	1024^2+	CLIP-L+T5	Hybrid DiT	최고 품질	~16-20GB
Flux.1 Schnell	12B	1024^2+	CLIP-L+T5	Hybrid DiT	초고속 (3-6 스텝)	~12-16GB
Flux.2 Dev	32B	4MP	Mistral 3.1	Flow Matching	World knowledge	~20-24GB
Qwen Image	20B	다양	MMDiT	MMDiT	다국어 텍스트 최고	~4GB+

---

8. LoRA와 파인튜닝

8.1 LoRA 개념

Full Name

LoRA: Low-Rank Adaptation

개념

LoRA는 대규모 사전학습 모델을 효율적으로 파인튜닝하는 기법입니다. 전체 모델 가중치를 업데이트하는 대신, **저랭크(low-rank) 행렬 분해**를 사용하여 훨씬 적은 파라미터만 학습합니다.

수식

```
W' = W₀ + ΔW = W₀ + β · B · A

여기서:
- W₀: 원본 고정 가중치 (frozen)
- ΔW: 학습 가능한 업데이트 (B·A 저랭크 분해)
- A: 랭크 r의 다운 프로젝션 행렬 (d × r)
- B: 랭크 r의 업 프로젝션 행렬 (r × d)
- β: 스케일링 팩터 (보통 α/r, α는 하이퍼파라미터)
- r: 랭크 (일반적으로 4, 8, 16, 32, 64)
```

예시

```
원본 가중치: 1024 × 1024 = 1,048,576 파라미터
LoRA (rank=8):
  - A: 1024 × 8 = 8,192 파라미터
  - B: 8 × 1024 = 8,192 파라미터
  - 총: 16,384 파라미터 (약 1/64로 감소)
```

8.2 LoRA 장점

1. 메모리 효율성

- 전체 파인튜닝 대비 수백 배 적은 파라미터
- 소비자 GPU에서 파인튜닝 가능

2. 빠른 학습

- 더 적은 파라미터 → 더 빠른 수렴
- 적은 데이터로도 효과적

3. 모듈성

- 여러 LoRA 어댑터 생성 가능
- 베이스 모델에 동적으로 병합

4. 저장 공간

- LoRA 파일: 일반적으로 10-200MB

- 전체 모델: 수 GB
- 여러 스타일/개념을 작은 파일로 저장

5. 조합 가능

- 여러 LoRA를 동시에 적용 가능
- 가중치 조절로 영향력 제어

8.3 LoRA 사용 사례

스타일 학습

- 특정 예술 스타일 (애니메이션, 유화, 수채화 등)
- 작가의 독특한 화풍

캐릭터/객체 학습

- 특정 인물, 캐릭터, 마스코트
- 제품, 로고, 브랜딩 요소

개념 학습

- 추상적 개념 (예: "드림코어", "사이버펑크")
- 특정 조명, 분위기

도메인 특화

- 의료 영상
- 건축 렌더링
- 패션 디자인

8.4 LoRA 파인튜닝 프로세스

1. 데이터 준비

- **이미지 수집**: 5-50장 (스타일) 또는 15-100장 (캐릭터)
- **캡셔닝**: 각 이미지에 상세한 텍스트 설명
- **전처리**: 크기 조정, 크롭, 정규화

2. 학습 설정

- **베이스 모델**: SD1.5, SDXL, Flux 등
- **랭크 (r)**: 4-64 (높을수록 표현력 증가, 메모리 증가)
- **Alpha (α)**: 보통 랭크와 동일

- **학습률**: 1e-4 ~ 1e-5
- **학습 스텝**: 500-5000

3. 학습 실행

- **방법**: DreamBooth, Textual Inversion, 또는 일반 LoRA
- **정규화**: Prior preservation loss (선택사항)
- **모니터링**: 과적합 방지

4. LoRA 저장

- **파일 형식**: .safetensors (권장) 또는 .pt
- **크기**: 일반적으로 10-200MB
- **위치**: `models/loras/`

8.5 LoRA 병합

가중치 합산 방식

```
W_merged = W₀ + β₁·B₁·A₁ + β₂·B₂·A₂ + ...

예시:
- 베이스 모델 (100%)
- + LoRA 1 (스타일) × 0.7
- + LoRA 2 (캐릭터) × 1.0
- + LoRA 3 (조명) × 0.5
```

ComfyUI에서 LoRA 사용

1. Load LoRA 노드 추가
2. LoRA 파일 선택
3. **강도 조절**: 0.0-2.0 (일반적으로 0.7-1.0)
4. 모델 및 CLIP에 연결

8.6 LoRA 변형

AdaLoRA

- **Full name**: Adaptive Low-Rank Adaptation
- **특징**: 학습 중 랭크 동적 조절
- **장점**: 더 효율적인 파라미터 사용

LoHa (Locon)

- **Full name**: Low-Rank Hadamard Adaptation
- **방법**: Hadamard product 사용
- **특징**: 더 복잡한 변환 가능

LyCORIS

- **Full name**: Lora beYond Conventional methods, Other Rank adaptation Implementations for Stable diffusion
- **특징**: LoRA의 일반화된 형태

8.7 LoRA 파일명 예시

```
anime_style_lora_sdxl_v1.safetensors          # SDXL 애니메이션 스타일
character_john_flux_r32.safetensors           # Flux 캐릭터, 랭크 32
cyberpunk_lighting_sd15_v2.safetensors        # SD1.5 사이버펑크 조명
```

8.8 실무 팁

최적 랭크 선택

- **스타일 LoRA**: rank=8-16 (충분)
- **캐릭터 LoRA**: rank=16-32 (디테일 중요)
- **복잡한 개념**: rank=32-64 (높은 표현력)

과적합 방지

- 충분한 이미지 다양성
- 정규화 기법 사용
- 조기 종료 (early stopping)

품질 검증

- 다양한 프롬프트로 테스트
- 강도 조절 실험 (0.5-1.5)
- 다른 LoRA와 조합 테스트

---

9. 실습 팁과 워크플로우 구성

9.1 ComfyUI 워크플로우 기본 구조

Text-to-Image 기본 워크플로우

```
Load Checkpoint
    ↓
CLIP Text Encode (Positive Prompt)
    ↓
CLIP Text Encode (Negative Prompt)
    ↓
Empty Latent Image (크기 설정)
    ↓
KSampler (샘플링)
    ↓
VAE Decode
    ↓
Save Image
```

Image-to-Image 워크플로우

```
Load Checkpoint
    ↓
Load Image
    ↓
VAE Encode → Latent
    ↓
CLIP Text Encode (프롬프트)
    ↓
KSampler (denoise < 1.0)
    ↓
VAE Decode
    ↓
Save Image
```

LoRA 적용 워크플로우

```
Load Checkpoint
    ↓
Load LoRA (강도 조절)
    ↓
CLIP Text Encode
    ↓
Empty Latent Image
    ↓
KSampler
    ↓
VAE Decode
```

```
      ↓
Save Image
```

9.2 KSampler 파라미터 이해

Steps (샘플링 스텝)

- **SD1.5/2.x**: 20-50 스텝
- **SDXL**: 20-40 스텝
- **SD3**: 20-30 스텝
- **Flux Dev**: 20-50 스텝
- **Flux Schnell**: 3-6 스텝 (초고속)
- **원칙**: 더 많은 스텝 = 더 정제된 이미지, 더 긴 시간

CFG Scale (Classifier-Free Guidance)

- **범위**: 1.0-20.0
- **일반적**: 7.0-8.0
- **낮음 (1-5)**: 창의적, 덜 제약적, 프롬프트 이탈 가능
- **중간 (7-10)**: 균형잡힌 준수도
- **높음 (15-20)**: 엄격한 프롬프트 준수, 과포화 위험
- **특수**: Flux Schnell, SD3 Turbo는 0으로 설정

Sampler (샘플러)

- **Euler a**: 빠르고 창의적
- **DPM++ 2M Karras**: 고품질, 안정적 (권장)
- **DPM++ SDE Karras**: 디테일 높음, 느림
- **DDIM**: 재현 가능, 결정론적
- **UniPC**: 매우 빠름
- **FlowMatchEulerDiscrete**: Flux 전용

Scheduler (스케줄러)

- **Karras**: 일반적 추천
- **Exponential**: 부드러운 전환
- **Simple**: 기본

Denoise (노이즈 제거 강도)

- **1.0**: 완전히 새로운 이미지 생성 (Text-to-Image)
- **0.5-0.8**: 원본 유지하면서 변형 (Image-to-Image)

- 0.3-0.5: 약간의 수정
- 0.1-0.3: 매우 미세한 조정

9.3 프롬프트 작성 가이드

효과적인 프롬프트 구조

```
[주제] + [스타일] + [디테일] + [품질 태그] + [기술 설정]

예시:
"A majestic lion, digital painting, volumetric lighting,
golden hour, highly detailed fur, 8k, trending on artstation,
sharp focus"
```

프롬프트 요소

1. 주제 (Subject)

- 명확하고 구체적으로
- 예: "a young woman with red hair" (❌ "a person")

2. 스타일 (Style)

- 예술 스타일, 매체
- 예: "oil painting", "anime style", "photorealistic", "3D render"

3. 조명 (Lighting)

- 예: "soft lighting", "dramatic shadows", "golden hour", "studio lighting"

4. 구도 (Composition)

- 예: "close-up portrait", "wide angle", "bird's eye view"

5. 품질 태그 (Quality Tags)

- 예: "highly detailed", "8k", "masterpiece", "sharp focus"

6. 아티스트 참조 (선택사항)

- 예: "by Greg Rutkowski", "in the style of Studio Ghibli"

네거티브 프롬프트 (Negative Prompt)

원하지 않는 요소를 제외합니다:

```
"blurry, low quality, distorted, deformed, ugly, bad anatomy,
watermark, text, logo, cropped, out of frame"
```

9.4 해상도 및 종횡비 가이드

Stable Diffusion 1.5/2.x

- **최적**: 512×512
- **지원**: 512×768, 768×512 (약간 품질 저하 가능)

SDXL

- **최적**: 1024×1024
- **지원**:
  - 1024×1536 (세로)
  - 1536×1024 (가로)
  - 1280×960
  - 960×1280

SD3 / Flux

- **최적**: 1024×1024 이상
- **유연성**: 다양한 종횡비 지원

종횡비 권장

- **1:1** (정사각형): 1024×1024
- **4:3** (가로): 1024×768
- **3:4** (세로): 768×1024
- **16:9** (와이드): 1024×576
- **9:16** (세로 와이드): 576×1024

9.5 VRAM 최적화 전략

저사양 GPU (6-8GB VRAM)

1. **모델 선택**: SD1.5, SDXL (FP16) 사용
2. **해상도 낮추기**: 512×512 또는 768×768
3. **배치 크기**: 1로 설정
4. **VAE Tiling**: 큰 이미지 분할 처리

5. **텍스트 인코더 오프로딩**: T5-XXL 제외 또는 FP8 사용

중급 GPU (10-12GB VRAM)

1. **모델 선택**: SDXL, SD3 (작은 변형)
2. **해상도**: 1024×1024 가능
3. **LoRA**: 여러 개 동시 사용 가능

고급 GPU (16-24GB VRAM)

1. **모델 선택**: SD3, Flux Dev, 모든 모델
2. **해상도**: 1024×1024 이상
3. **배치 생성**: 동시에 여러 이미지 가능

9.6 품질 향상 기법

1. Hires Fix (고해상도 수정)

- **방법**: 낮은 해상도로 생성 후 업스케일 + 재샘플링
- **효과**: 디테일 향상, artifact 감소
- **설정**: 업스케일 2x, denoise 0.4-0.6

2. ControlNet

- **목적**: 포즈, 깊이, 엣지 등으로 제어
- **종류**:
    - Canny: 엣지 검출
    - Depth: 깊이 맵
    - OpenPose: 포즈 제어
    - Scribble: 스케치 기반

3. Inpainting (인페인팅)

- **목적**: 이미지 일부 수정
- **프로세스**:
    1. 마스크 생성 (수정할 영역)
    2. 프롬프트 작성
    3. Inpainting 모델 사용
    4. denoise 0.6-1.0

4. Img2Img Refinement

- **목적**: 기존 이미지 개선

- **설정**: denoise 0.3-0.5

9.7 비디오 생성 워크플로우 (2026)

기본 Text-to-Video

```
Load Video Model (Kling/Wan/Veo)
    ↓
Text Prompt
    ↓
Video Generation Node
    ↓
Save Video
```

Image-to-Video

```
Load Image
    ↓
Load Video Model
    ↓
Image-to-Video Node (첫 프레임 또는 참조)
    ↓
Save Video
```

Element Video (Kling 1.6)

- 최대 4개 이미지 업로드
- 각 이미지에 대한 설명
- 모든 요소를 포함하는 비디오 생성

9.8 트러블슈팅

문제 1: 검은 이미지 생성

원인: VAE 오류, VRAM 부족 **해결**:

- 다른 VAE 시도
- 해상도 낮추기
- FP16 → FP32 VAE

문제 2: 프롬프트가 무시됨

원인: CFG Scale 너무 낮음, 잘못된 인코더 **해결**:

- CFG Scale 7-8로 설정
- 텍스트 인코더 확인 (CLIP, T5 로드 확인)

문제 3: 과포화/이상한 색상

원인: CFG Scale 너무 높음 **해결**:

- CFG Scale 7-10으로 낮추기

문제 4: Out of Memory (OOM)

원인: VRAM 초과 **해결**:

- 해상도 낮추기
- 배치 크기 1로
- FP16 모델 사용
- 텍스트 인코더 오프로딩

문제 5: 느린 생성 속도

원인: CPU 병목, 비효율적 샘플러 **해결**:

- DPM++ 2M Karras 샘플러 사용
- 스텝 수 감소
- Flux Schnell 사용 (3-6 스텝)

9.9 실습 프로젝트 아이디어

초급

1. **기본 이미지 생성**: 다양한 프롬프트로 실험
2. **스타일 변형**: 같은 주제, 다른 스타일
3. **LoRA 적용**: 애니메이션 스타일 LoRA 사용

중급

1. **캐릭터 일관성**: LoRA로 특정 캐릭터 학습 및 생성
2. **Image-to-Image 변환**: 사진을 예술 작품으로
3. **Inpainting**: 이미지 일부 수정

고급

1. **커스텀 LoRA 학습**: 자신만의 스타일/캐릭터
2. **ControlNet 활용**: 포즈, 깊이 제어 이미지 생성
3. **비디오 생성**: 텍스트 또는 이미지로 짧은 비디오 제작

4. **오디오-비디오 통합**: Kling 2.6으로 사운드 포함 비디오

---

10. 참고 자료 및 커뮤니티

10.1 공식 문서

- **ComfyUI**: https://github.com/comfyanonymous/ComfyUI
- **ComfyUI 문서**: https://docs.comfy.org/
- **Stable Diffusion**: https://stability.ai/
- **Hugging Face**: https://huggingface.co/

10.2 모델 다운로드

- **Civitai**: https://civitai.com/ (LoRA, 체크포인트)
- **Hugging Face Models**: https://huggingface.co/models
- **Stability AI**: https://huggingface.co/stabilityai

10.3 커뮤니티

- **Reddit**: r/StableDiffusion, r/comfyui
- **Discord**: ComfyUI 공식 Discord
- **GitHub**: ComfyUI Issues 및 Custom Nodes

10.4 학습 리소스

- **YouTube**: ComfyUI 튜토리얼
- **Medium/Blog**: AI 아트 가이드
- **Civitai**: 프롬프트 및 설정 공유

---

11. 용어 색인 (Glossary)

A-C

- **ACE-Step**: A Step Towards Music Generation Foundation Model
- **AdaLN**: Adaptive Layer Normalization
- **API**: Application Programming Interface
- **BF16**: Brain Floating Point 16
- **CLIP**: Contrastive Language-Image Pre-training
- **CLIP-G**: CLIP Giant (1.4B parameters)

- **CLIP-H**: CLIP Huge (632M parameters)
- **CLIP-L**: CLIP Large (427M parameters)
- **CFG**: Classifier-Free Guidance

D-G

- **DCAE**: Deep Compression AutoEncoder
- **DiT**: Diffusion Transformer
- **FP8**: 8-bit Floating Point
- **FP16**: 16-bit Floating Point (Half Precision)
- **FP32**: 32-bit Floating Point (Single Precision)
- **FP64**: 64-bit Floating Point (Double Precision)
- **GGUF**: GPT-Generated Unified Format
- **GPU**: Graphics Processing Unit

H-M

- **INT8**: 8-bit Integer
- **LLM**: Large Language Model
- **LoRA**: Low-Rank Adaptation
- **MERT**: Music Understanding Model
- **MMDiT**: Multimodal Diffusion Transformer
- **MoE**: Mixture of Experts

N-S

- **ODE**: Ordinary Differential Equation
- **REPA**: Representation Alignment using MERT and m-hubert
- **RF**: Rectified Flow
- **RoPE**: Rotary Positional Embeddings
- **SD**: Stable Diffusion
- **SDXL**: Stable Diffusion XL
- **SD3**: Stable Diffusion 3

T-Z

- **T5**: Text-To-Text Transfer Transformer
- **T5-XXL**: T5 Extra Extra Large (11B parameters)
- **TF32**: TensorFloat-32
- **UMT5**: Universal Multilingual T5
- **UMT5-XXL**: Universal Multilingual T5 - Extra Extra Large

- **UNet**: U-shaped Network
- **VAE**: Variational Autoencoder
- **VRAM**: Video Random Access Memory

양자화 관련

- **Q2_K**: 2-bit K-quantization
- **Q3_K_M**: 3-bit K-quantization, Medium
- **Q4_K_M**: 4-bit K-quantization, Medium (most popular)
- **Q4_K_S**: 4-bit K-quantization, Small
- **Q5_K_M**: 5-bit K-quantization, Medium
- **Q6_K**: 6-bit K-quantization
- **Q8_0**: 8-bit Linear Quantization

마무리

이 강의노트는 ComfyUI와 오픈소스 생성형 AI 모델을 활용한 콘텐츠 생성의 핵심 개념을 다룹니다. 2026년 2월 기준 최신 정보를 반영하였으며, 지속적으로 발전하는 분야이므로 공식 문서와 커뮤니티를 통해 최신 정보를 확인하시기 바랍니다.

실습 시작 전 체크리스트:

- ☐ ComfyUI 최신 버전 설치
- ☐ 필요한 모델 다운로드 (체크포인트, VAE, 텍스트 인코더)
- ☐ GPU 드라이버 최신 업데이트
- ☐ 충분한 저장 공간 확보 (모델 파일 크기 고려)
- ☐ 기본 워크플로우 이해 및 실습

학습 목표:

- ✅ 수치 정밀도의 차이와 선택 기준 이해
- ✅ 모델 파일 형식의 특징과 사용법 숙지
- ✅ 텍스트 인코더의 역할과 조합 전략 파악
- ✅ VAE의 작동 원리 이해
- ✅ 양자화 기법과 실용적 선택 능력
- ✅ 주요 생성 모델의 아키텍처 이해
- ✅ LoRA 파인튜닝 기초 습득
- ✅ ComfyUI 워크플로우 구성 능력

다음 단계:

1. 기본 Text-to-Image 워크플로우 실습

2. 다양한 샘플러와 파라미터 실험
3. LoRA 적용 및 조합 실습
4. Image-to-Image 및 Inpainting 워크플로우
5. (선택) 비디오 생성 실습
6. (선택) 오디오 생성 실습